

Docker 1단계: 개념

3개월 자기주도 BE 전환 프로그램 · 시작 전 준비 · 학습 노트 · 2026-09-08

이 문서는 개념만 다룬다. 명령어는 2단계, Compose 문법은 3단계에서 이어진다. 끝의 확인 질문 3개에 답한 뒤 2단계로 넘어간다.

1. Docker가 해결하는 문제

서버 프로그램은 혼자 돌지 않는다. MySQL은 특정 리눅스 라이브러리, 설정 파일, 데이터 디렉터리, 포트가 필요하다. 이것을 사람 손으로 설치하면 컴퓨터마다 상태가 달라지고, 몇 달 뒤엔 어떤 설정을 건드렸는지 아무도 모르게 된다.

Unity로 치면 이렇다. 팀원 A의 프로젝트는 되고 B는 안 되는데, 알고 보니 B의 Unity 버전이나 패키지가 달랐던 경험이 있을 것이다. Unity Hub가 버전을 고정해 주듯, Docker는 **서버 프로그램의 실행 환경 전체를 고정해 준다**.

핵심 약속은 하나다. **"같은 이미지면 어디서 실행해도 같은 결과."**

2. 이미지와 컨테이너

이 둘의 구분이 Docker의 전부라고 해도 된다.

이미지(image)는 실행에 필요한 모든 것을 담은 읽기 전용 파일 묶음이다. 리눅스 최소 파일 시스템 + MySQL 바이너리 + 기본 설정 + "시작할 때 이 명령을 실행하라"는 지시까지 들어 있다. 이미지는 절대 바뀌지 않는다.

컨테이너(container)는 이미지를 실제로 실행한 프로세스다. 이미지 위에 얇은 "쓰기 가능한 층"을 얹어서 실행 중에 생기는 변경(로그, 임시 파일)을 거기에 쓴다. 컨테이너를 지우면 그 층도 사라지고, 이미지는 그대로 남는다.

C#에 대응하면 이미지는 클래스, 컨테이너는 인스턴스다. 한 이미지로 컨테이너를 여러 개 만들 수 있고, 각 컨테이너는 서로 격리된다. `mysql:9.7` 이미지 하나로 테스트용 DB 세 개를 띄울 수 있다.

3. 이미지 이름과 태그

이미지는 `이름:태그` 형식으로 부른다. `mysql:9.7` 에서 `mysql` 이 이름, `9.7` 이 태그(버전)다. 태그를 생략하면 `latest` 가 붙는데, **실무에서는 절대 `latest` 를 쓰지 않는다**. 어느 날 갑자기 다른 버전이 내려와서 동작이 바뀌기 때문이다. VERSIONS.md에 버전을 고정한 이유와 같다.

이미지는 **레지스트리**에서 받아온다. 기본 레지스트리는 Docker Hub이고, `mysql`, `opensearchproject/opensearch` 처럼 공식 또는 벤더가 관리하는 이미지가 있다. Unity Asset Store에서 패키지를 내려받는 것과 같은 구조다. 회사에서는 보통 사내 레지스트리를 따로 둔다.

4. 레이어

이미지는 한 덩어리가 아니라 **레이어**가 쌓인 구조다. 예를 들어 `mysql:9.7` 은 "리눅스 기본 파일" 레이어 위에 "MySQL 설치" 레이어, "설정" 레이어가 올라간 형태다.

이것이 중요한 이유는 두 가지다.

- 같은 기반 레이어를 쓰는 이미지끼리는 그 레이어를 공유하므로 디스크와 다운로드 시간이 절약된다.
- 나중에 Spring 앱 이미지를 만들 때(4주 차 BE-012), "JDK 레이어"는 그대로 두고 "내 jar 파일 레이어"만 바꾸면 빌드가 빨라진다. Unity의 AssetBundle 의존성 구조와 비슷한 발상이다.

5. 컨테이너와 가상 머신의 차이

가상 머신은 OS 커널까지 통째로 띄운다. 그래서 무겁고 부팅에 수십 초가 걸린다.

컨테이너는 **호스트의 리눅스 커널을 공유**하고, 커널 기능(namespace, cgroup)으로 프로세스·파일 시스템·네트워크·자원 사용량만 격리한다. 그래서 일반 프로세스처럼 1초 안에 뜨고 메모리도 적게 쓴다.

Mac 사용자가 꼭 알아야 할 점. 컨테이너는 리눅스 커널 기능이므로 macOS에서 직접 돌 수 없다. Docker Desktop은 내부에 아주 작은 리눅스 가상 머신을 하나 띄우고, 그 안에서 컨테이너를 실행한다. 평소엔 의식할 일이 없지만 다음 두 상황에서 차이가 드러난다.

- Mac 폴더를 컨테이너에 연결하면 VM을 거치므로 파일 접근이 느리다. DB 데이터는 Mac 폴더 대신 Docker가 관리하는 볼륨(7번)에 두는 것이 낫다.
- 컨테이너 안에서 "localhost"는 그 컨테이너 자신을 뜻하지, 내 Mac이 아니다.

6. 포트 매핑

컨테이너는 자기만의 네트워크 공간을 가진다. 컨테이너 안의 MySQL이 3306 포트를 열어도, 밖(내 Mac)에서는 보이지 않는다. 그래서 "Mac의 3306 → 컨테이너의 3306"처럼 **포트 매핑**을 선언해야 Spring 앱이 `localhost:3306`으로 접속할 수 있다.

Mac 쪽 포트 번호는 자유롭게 정할 수 있다. 이미 Mac에 다른 MySQL이 3306을 쓰고 있다면 "Mac의 13306 → 컨테이너의 3306"으로 잡아도 된다. 컨테이너 안 프로그램은 자기 포트가 밖에서 몇 번으로 보이는지 모른다.

7. 볼륨

컨테이너를 지우면 쓰기 층이 함께 사라진다고 했다. MySQL이 저장한 테이블 데이터도 컨테이너 안 `/var/lib/mysql`에 있으므로, 컨테이너를 지우면 **DB가 통째로 날아간다**.

이를 막는 것이 **볼륨**이다. 컨테이너 안의 특정 경로를 컨테이너 밖 저장소에 연결해서, 컨테이너가 죽거나 재생성돼도 데이터가 남게 한다. 볼륨에는 두 종류가 있다.

- **named volume:** Docker가 관리하는 저장 공간. 이름만 붙여 두면 된다. DB 데이터에 권장.
- **bind mount:** 내 Mac의 특정 폴더를 직접 연결. 설정 파일이나 초기화 SQL 스크립트를 넣을 때 쓴다.

Unity에 빗대면 컨테이너의 쓰기 층은 PlayerPrefs 없이 메모리에만 있는 상태고, 볼륨은 `persistentDataPath`에 저장하는 것이다.

8. 환경변수로 설정 주입

이미지는 바뀌지 않으므로, 비밀번호나 DB 이름 같은 설정은 이미지에 굽지 않고 **실행할 때 환경변수로 넣는다**. MySQL 이미지는 `MYSQL_ROOT_PASSWORD`, `MYSQL_DATABASE` 같은 환경변수를 읽어서 처음 시작할 때 계정과 DB를 만든다.

이 방식이 커리큘럼의 "비밀정보를 코드에 넣지 않는다"는 규칙과 연결된다. 비밀번호는 `.env` 파일에만 두고, 그 파일은 `.gitignore` 로 제외한다. 이미 `.env.example` 에 키 이름만 적어 둔 이유다.

9. 컨테이너의 수명

컨테이너는 "시작 명령으로 지정된 프로세스 하나"가 살아 있는 동안만 산다. MySQL 컨테이너는 `mysqld` 프로세스가 곧 컨테이너이고, 그 프로세스가 종료되면 컨테이너도 종료된다. 서버가 아니라 일회성 명령(예: 마이그레이션 스크립트)을 컨테이너로 돌리면 명령이 끝나는 즉시 컨테이너가 끝난다.

이 성질 때문에 "컨테이너 안에 들어가서 설정을 고친다"는 접근은 잘못된 습관이다. 고친 내용은 컨테이너를 재생성하면 사라진다. 바꾸고 싶은 것은 환경변수, 볼륨, 또는 이미지 자체로 표현해야 한다.

1단계 정리

개념	한 줄 정의	비유
이미지	읽기 전용 실행 환경 묶음	클래스, 빌드된 .app
컨테이너	이미지를 실행한 격리 프로세스	인스턴스
태그	이미지의 버전 표시	패키지 버전 고정
레지스트리	이미지 저장소	Asset Store
포트 매핑	호스트 포트를 컨테이너 포트에 연결	방화벽 포트 포워딩
볼륨	컨테이너 밖에 남는 데이터 저장소	<code>persistentDataPath</code>
환경변수	실행 시점에 넣는 설정	빌드 파라미터

확인 질문 (답을 적은 뒤 2단계로)

1. `mysql:9.7` 컨테이너를 띄워서 테이블을 만들고 데이터를 넣은 뒤, 컨테이너를 삭제하고 같은 이미지로 다시 띄웠다. 볼륨을 잡지 않았다면 데이터는 어떻게 될까? 그 이유를 이미지·컨테이너·쓰기 총 개념으로 설명해 보라.

2. 회사에서 `mysql:latest` 를 쓰지 말라고 한다. 어떤 사고가 날 수 있는가?

3. Spring 앱을 내 Mac에서 직접 실행하고 MySQL만 컨테이너로 띄웠다. 앱의 DB 접속 주소를 `localhost:3306` 으로 쓰면 되는 이유는 무엇이고, 그렇게 되려면 무엇을 선언해야 하는가?